

MVC PHP TD4

Sommaire

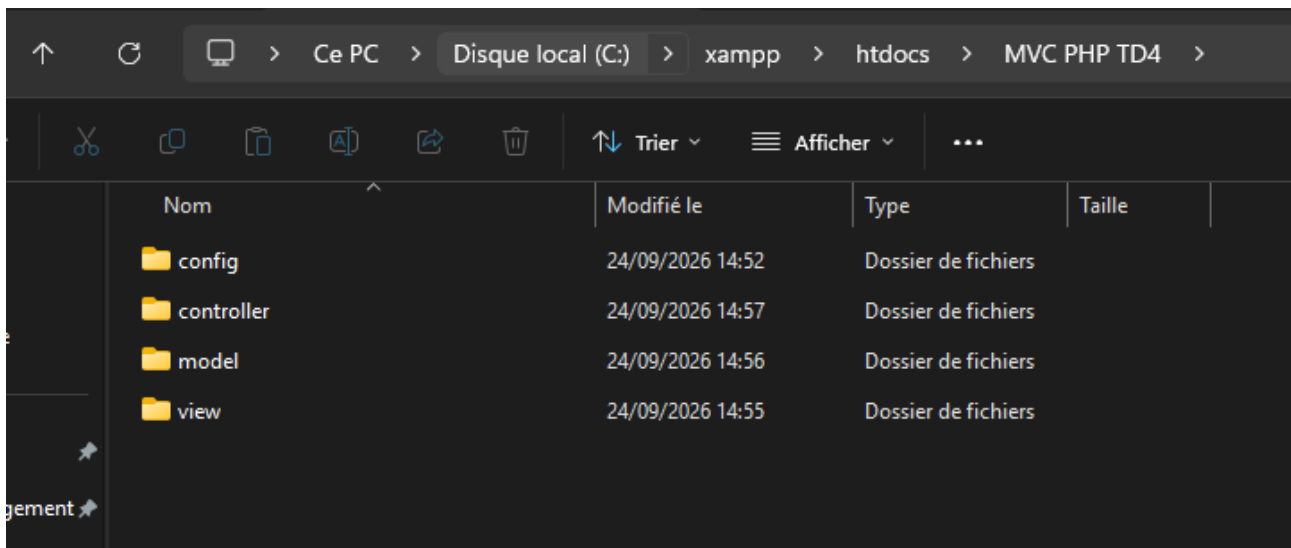
<u>Introduction</u>	1
<u>Exercice:1</u>	2
<u>Exercice :2</u>	2
<u>Exercice:3</u>	4
<u>Exercice :4</u>	5
<u>Exercice:5</u>	6
<u>Exercice : 6</u>	6
<u>Exercice:7</u>	7
<u>Exercice : 8</u>	8
<u>Exercice : 9</u>	8
<u>Conclusion</u>	8

Introduction

Dans ce TD, nous allons continuer à travailler avec PHP et une base de données. Nous allons notamment voir les requêtes préparées pour éviter les injections SQL, puis mettre en place des relations entre plusieurs tables avec les clés étrangères et les jointures SQL. Le but est de mieux comprendre comment sécuriser et organiser les données d'une application.

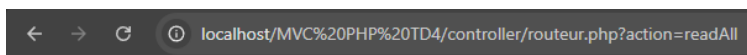
Exercice:1

J'ai commencé par créer les dossiers config, controller, model et view. J'ai ensuite déplacé les fichiers du TD précédent dans les bons dossiers. J'ai renommé Voiture.php en ModelVoiture.php et j'ai adapté les chemins des fichiers pour que le projet fonctionne avec la nouvelle architecture MVC.



Exercice :2

J'ai créé la vue list.php. Elle permet d'afficher la liste des voitures récupérées dans la base de données. La vue sert seulement à afficher les informations et ne fait pas les requêtes SQL.



Liste des voitures

Immatriculation : [AA111BB](#) | Marque : renault | Couleur : rouge
Immatriculation : [AB123CD](#) | Marque : Mercedes | Couleur : Noir
Immatriculation : [EF456GH](#) | Marque : BMW | Couleur : Blanc
Immatriculation : [IJ789KL](#) | Marque : Renault | Couleur : Rouge
Immatriculation : [MN123OP](#) | Marque : Ferrari | Couleur : Jaune
Immatriculation : [QR456ST](#) | Marque : Audi | Couleur : Gris
Immatriculation : [UV789WX](#) | Marque : Peugeot | Couleur : Bleu
Immatriculation : [ZZ999ZZ](#) | Marque : Toyota | Couleur : Vert

Exercice:3

J'ai créé ControllerVoiture.php. Le contrôleur récupère les voitures grâce au modèle puis envoie les résultats à la vue list.php. Cela permet de séparer le traitement de l'affichage.

```

}

// Afficher le formulaire de création
public static function create() {
    require __DIR__ . '/../view/voiture/create.php';
}

// Créer une voiture
public static function created() {
    $immatriculation = $_GET['immatriculation'];
    $marque = $_GET['marque'];
    $couleur = $_GET['couleur'];

    $v = new ModelVoiture(
        $marque,
        $couleur,
        $immatriculation
    );

    $v->save();
    self::readAll();
}
}
?>
```

Exercice :4

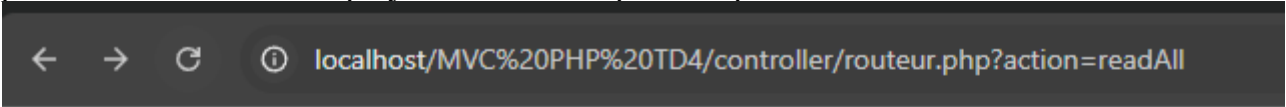
J'ai créé le fichier routeur.php. Il permet d'appeler l'action readAll du contrôleur. Grâce au routeur, je n'ai plus besoin d'appeler directement le contrôleur.

```

6
7     $action = $_GET['action'];
8
9     if ($action == 'readAll') {
10         ControllerVoiture::readAll();
11     }
12
13     elseif ($action == 'read') {
14         ControllerVoiture::read();
15     }
16
17     elseif ($action == 'create') {
18         ControllerVoiture::create();
19     }
20
21     elseif ($action == 'created') {
22         ControllerVoiture::created();
23     }
24
25     else {
26         echo "Action inconnue : " . htmlspecialchars($action);
27     }
28
29 } else {
30
31     ControllerVoiture::readAll();
32
33 }
34
35 ?>
36
```

Exercice:5

J'ai modifié le routeur pour récupérer l'action directement dans l'URL avec \$_GET['action']. Cela permet de choisir l'action que je veux exécuter, par exemple readAll ou create.



← → ↻ ⓘ localhost/MVC%20PHP%20TD4/controller/routeur.php?action=readAll

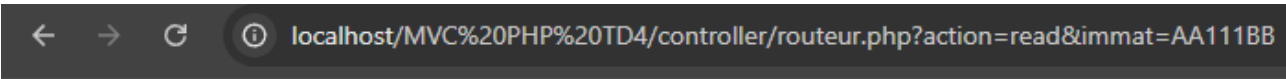
Liste des voitures

Immatriculation : [AA111BB](#) | Marque : renault | Couleur : rouge

Immatriculation : [AB123CD](#) | Marque : Mercedes | Couleur : Noir

Exercice : 6

J'ai créé une page de détail pour afficher toutes les informations d'une voiture. Je récupère son immatriculation dans l'URL, puis le modèle recherche la voiture dans la base de données. J'ai aussi ajouté un message d'erreur si l'immatriculation n'existe pas.



← → ↻ ⓘ localhost/MVC%20PHP%20TD4/controller/routeur.php?action=read&immat=AA111BB

Détail de la voiture

Immatriculation : AA111BB

Marque : renault

Couleur : rouge

[Retour à la liste](#)

Exercice:7

J'ai créé un formulaire pour ajouter une voiture. Les informations saisies sont envoyées au contrôleur avec GET. Le contrôleur crée ensuite une nouvelle voiture et utilise la méthode save() pour l'enregistrer dans la base de données. Après l'ajout, je retourne automatiquement à la liste.



← → ↻ ⓘ localhost/MVC%20PHP%20TD4/controller/routeur.php?action=create

Ajouter une voiture

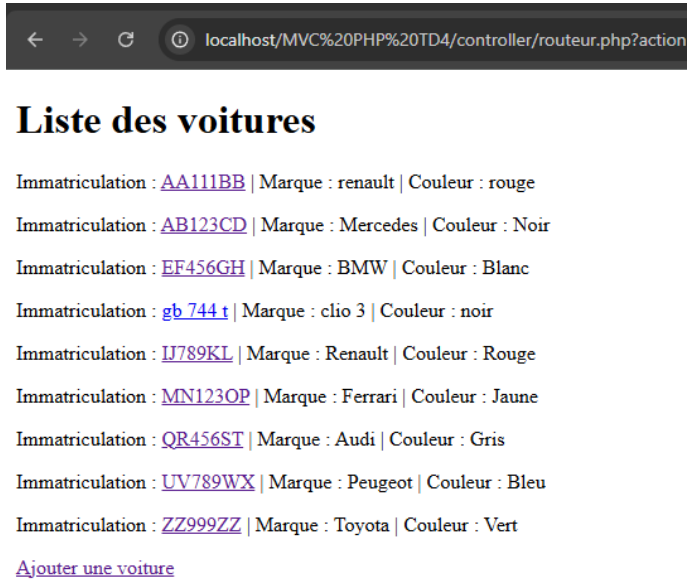
Immatriculation :

Marque :

Couleur :

[Retour à la liste](#)

Après avoir ajouter une voiture : (clio 3 noir)



← → ↻ ⓘ localhost/MVC%20PHP%20TD4/controller/routeur.php?action=...

Liste des voitures

Immatriculation : [AA111BB](#) | Marque : renault | Couleur : rouge

Immatriculation : [AB123CD](#) | Marque : Mercedes | Couleur : Noir

Immatriculation : [EF456GH](#) | Marque : BMW | Couleur : Blanc

Immatriculation : [gb 744 t](#) | Marque : clio 3 | Couleur : noir

Immatriculation : [IJ789KL](#) | Marque : Renault | Couleur : Rouge

Immatriculation : [MN123OP](#) | Marque : Ferrari | Couleur : Jaune

Immatriculation : [QR456ST](#) | Marque : Audi | Couleur : Gris

Immatriculation : [UV789WX](#) | Marque : Peugeot | Couleur : Bleu

Immatriculation : [ZZ999ZZ](#) | Marque : Toyota | Couleur : Vert

[Ajouter une voiture](#)

ALOUANI Fadi

La voiture apparaît bien dans la Base de données .

Showing rows 0 - 8 (9 total, Query took 0.0003 seconds.) [marque: AUDI... - TOYOTA...]

```
SELECT * FROM `voiture` ORDER BY `marque` ASC
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort

Extra options

				immatriculation	marque	1	couleur
☐				QR456ST	Audi		Gris
☐				EF456GH	BMW		Blanc
☐				gb 744 t	clio 3		noir
☐				MN123OP	Ferrari		Jaune
☐				AB123CD	Mercedes		Noir
☐				UV789WX	Peugeot		Bleu
☐				AA111BB	renault		rouge
☐				IJ789KL	Renault		Rouge
☐				ZZ999ZZ	Toyota		Vert

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Sort

Conclusion

Ce TD nous a permis de mieux comprendre comment utiliser PHP avec une base de données. Nous avons vu l'intérêt des requêtes préparées pour sécuriser les données, ainsi que les clés étrangères et les jointures pour relier plusieurs tables. Ces notions sont importantes pour créer une application plus sécurisée et mieux organisée.